UW-TIG: Uncertainty-Weighted Temporal Information Gain for Closed-Loop Drone Defect Inspection, Evaluated in a Scalable Simulation Testbed

Akhil Ramalingam
Dept. of CSE
PSG College of Technology
Coimbatore, India

Anbuchandiran K
Dept. of CSE
PSG College of Technology
Coimbatore, India

Neelesh Padmanabh
Dept. of CSE
PSG College of Technology
Coimbatore, India

Saumiyaa Sri V L
Dept. of CSE
PSG College of Technology
Coimbatore, India

Sowndarya Elangi S
Dept. of CSE
PSG College of Technology
Coimbatore, India

Therdhana J P
Dept. of CSE
PSG College of Technology
Coimbatore, India

***Abstract*—Most drone inspection planners choose where to look from geometry alone. Such a planner has no idea what a defect is, carries nothing over from one flight to the next, and does not return for a second look when a detection is shaky. We describe UW-TIG, a viewpoint planner that extends a cost-normalised information-gain score with five ideas: a coverage phase, detector uncertainty, a defect memory that survives between missions and tracks growth, a staleness bonus for neglected spots, and a two-step lookahead, and a simulation testbed built so that the planner can be run, compared and repeated without a physical drone. In a nine-wall yard UW-TIG reaches precision 0.949, recall 0.967 and F1 0.953 on held-out seeds, level with the strongest of five re-implemented planners while flying 39% less. Growing the yard to 31 walls lowers every planner's score: UW-TIG reaches F1 0.921, statistically level with two rivals and ahead of an uncertainty-driven IPP rule with 44% less flight. Ablations credit the coverage phase with the gain; switching the uncertainty and growth terms off moves no metric yet. Fine-tuning the detector on two more public datasets lifted its held-out dacl10k mAP50 from 0.004 to 0.184 but did not help the closed loop. A synthetic oracle detector and a sweep of the uncertainty and growth weights from 0 to 20 confirm that those two terms change nothing in this simulator, and a like-for-like comparison of YOLOv8n, s and m shows the small model captures almost all of the size gain. We report these negative results alongside the positive ones.**

***Index Terms*—UAV inspection, next-best-view, informative path planning, defect detection, uncertainty, simulation.**

I. INTRODUCTION

Cracks, corrosion and leaks in buildings, towers and pipelines are found by people walking or flying a route and looking. Drones have made the looking cheaper, but the flying is usually still fixed: a pre-planned path, or a path that maximises how much surface is covered. What the camera sees does not change where the drone goes next.

Recent planners do close this loop in part. Uncertainty-driven informative path planning chooses views that most reduce a model's own doubt [1, 2, 3]; inspection planners route a UAV over target surfaces and add a defect detector [4, 5, 6]. Two gaps remain. These systems plan within a single session, so a defect found last month is not tracked or compared this month; and they report results on one scene size, so it is unclear whether a planner that works on nine walls still works on thirty.

We make three contributions. (1) **UW-TIG**, a planner that combines five ingredients in one score: a coverage phase, detector uncertainty, defect growth from persistent memory, staleness and a two-step lookahead. (2) A **reproducible simulation testbed** in which real defect photographs are the wall textures, with a nine-wall and a 31-wall yard, four scenarios and an evaluation protocol with separate development and test seeds. (3) An **evidence-first evaluation**: seven planners, six ablations, bootstrap intervals over seeds, ten seeds on the small yard, a second yard size, planner-only tests with a synthetic oracle detector, and a detector trained on extra data. Several of our own mechanisms turn out not to matter on the metrics we measure, and we say so.

II. RELATED WORK

Informative path planning. Rückin *et al.* choose UAV paths that teach a semantic-mapping network the most, scoring views by the network's own epistemic uncertainty per unit of travel cost [1]. Moon *et al.* reuse earlier planning effort for fast online replanning [2], and Vashisth *et al.* learn the planner with reinforcement learning on dynamic graphs [3]. All three work over a generic belief map, not over defects with an identity and a history.

Inspection planning. GATSBI routes a UAV over bridge surfaces with a travelling-salesman formulation and, in its later version, detects defects [4]. ASIP-Planner and FC-Vision handle obstacles that were missing from the map when the scan was planned [5, 6]; MOAR adapts paths to weather and battery [7], and Svystun *et al.* adapt wind-turbine inspections to blade pitch [8]. None of these revisits a defect across missions.

Classical building blocks. Cost-normalised entropy gain [9], receding-horizon next-best-view [10] and maximum-latency persistent monitoring [11] are the three older ideas we borrow; we re-implement all three as baselines.

Defect data. MBDD2025 is a set of 14,471 drone photos of building surfaces labelled with five defect classes [12]. dacl10k collects real bridge-inspection photographs [13], and its 2025 follow-up shows synthetic textures improve robustness [14]. Roboflow-100 is a public multi-domain detection benchmark [15]. Recent YOLO crack detectors report strong scores on single-class data [16].

III. SIMULATION TESTBED

The testbed has five parts, drawn in Fig. 1: a planner, a simulated camera, a trained detector, a geometry step that turns a detection into a position on a known wall, and a defect memory. Physics and the camera come from gym-pybullet-drones [17].

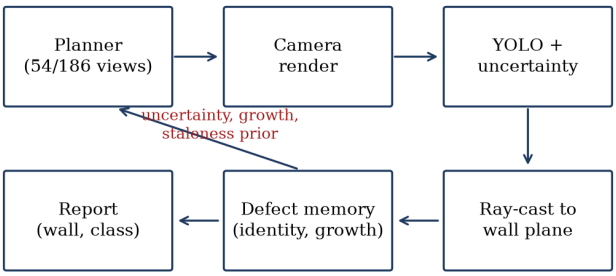


Fig. 1. The closed loop. The memory feeds uncertainty, growth and staleness back into the next planning step.

Yard. The original yard has nine inspectable walls: two freestanding buildings and a panel on each of a pipe rack and a lattice tower. The large yard has 31 (Fig. 2): four more buildings and six more freestanding panels over roughly 34 m by 28 m. Walls are flat textured quads, so ray-casting a detection onto a wall is exact. A wall that carries a defect gets a real MBDD2025 photo as its texture, cut to the wall's own width-to-height ratio so the defect is not stretched.

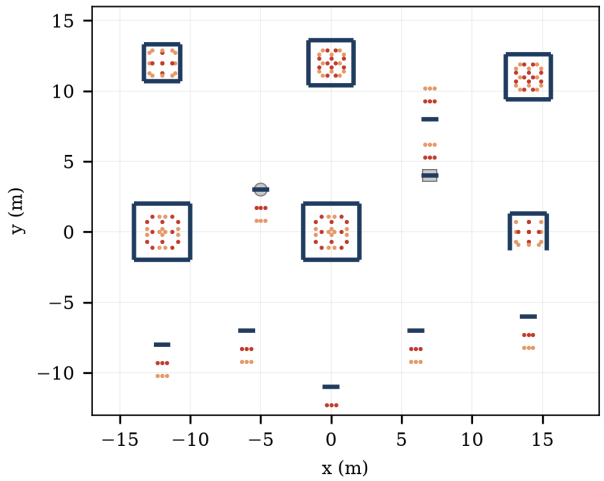


Fig. 2. The 31-wall yard (blue) and its 186 candidate viewpoints at 1.3 m (red) and 2.2 m (orange) standoff.

Viewpoints and scenarios. Each wall has six candidate viewpoints (two standoffs, three lateral offsets), 54 in the small yard and 186 in the large. Four scenarios place defects: static, uncertain (faint), multi-defect, and growing (a procedural defect whose size rises across missions). In the large yard the defect walls are drawn at random per seed. A mission is a fixed step budget (16 steps small, 50 large); each scenario runs three missions in a row so that memory persists.

Two ways to fly. A fast kinematic mode teleports the camera to each viewpoint and is used for all statistics; a PID-flight mode with full quadrotor dynamics is used for demonstration videos. We chose PyBullet over Unity or Unreal because the camera stays framed on the wall under inspection, so a more photorealistic renderer would not change what the detector sees [18].

IV. THE UW-TIG PLANNER

At each step the planner scores every candidate viewpoint v and flies to the best one:

$$U(v) = 1.0 \cdot IG(v) + 1.5 \cdot Unc(v) + 2.0 \cdot Gr(v) + 1.0 \cdot St(v) - 0.4 \cdot Cost(v) \quad (1)$$

where IG, Unc, Gr and St are the information-gain, uncertainty, growth and staleness terms described next and Cost is the travel distance. The weights were set once on development seeds and not tuned afterwards.

Information gain counts how much of the wall a viewpoint can still teach us. Walls are split into 1 m cells; a cell seen n times has belief $1 - 0.5e^{-0.6n}$ of being adequately observed, and IG(v) sums the entropy of the cells the viewpoint sees. **Uncertainty** is the spread of the detector's confidence over five copies of each frame (the original and four brightness, noise and blur variants); memory stores the last value per cell, and Unc(v) sums it, faded by the same $e^{-0.6n}$ so a spot does not pay out forever. **Growth** is the non-negative increase in a matched defect's physical size since the previous mission. **Staleness** is $1 - e^{-0.15t}$ per cell, with t the steps since the cell was last seen.

Coverage phase. While any wall is still unvisited in the current mission, the planner considers only the viewpoints of unvisited walls and scores them with IG(v) − 0.4 Cost(v) alone. It exists because a plain information-gain planner, and every early version of ours, stopped at four of nine walls on every seed. Under a cost-divided greedy rule the trip to the far building never pays for itself while nearby gain remains. No weight change fixed that; a separate phase did.

Lookahead. Once every wall has been seen, a candidate v_1 is credited with its own utility plus half the best utility available from the view that would follow it. Only v_1 is flown; the planner then replans from there.

Perception and memory. A YOLOv8s detector trained on MBDD2025 finds defects; each box centre is ray-cast onto the wall plane to get a 3-D position and physical size. A detection is matched to a stored defect if it has the same class and wall and lies within 0.35 m, using nearest neighbours of an appearance-and-position embedding; otherwise a new defect is created. A (wall, class) pair counts as *reported* once it has been seen three times (across missions) or once with confidence of at least 0.45; every planner is held to the same rule.

V. EXPERIMENTAL SETUP

Protocol. Viewpoint standoffs, the reporting rule and all bug fixes were settled on development seeds 100–102. Reported numbers come from a single run on test seeds 0–4. We report precision, recall and F1 of confirmed (wall, class) reports against ground truth, total flight distance, and coverage. Averages are over the static, uncertain and multi-defect scenarios; growing is excluded because the detector, trained on photographs, does not recognise its procedural defect. Error bars are 95% bootstrap intervals that resample the five seeds (each the mean over its scenarios) because rows from one seed are correlated.

Baselines. Seven planners share the viewpoint graph, belief and reporting rule: random choice; Isler-style cost-normalised entropy gain [9]; Bircher-style depth-two receding horizon [10]; Alamdari-style greedy maximum-latency [11]; a GATSBI-style tour over least-inspected walls [4]; an Rückin-style acquisition of uncertainty over visit count [1]; and UW-TIG. Each is our reading of the cited paper's *view-selection rule* only; we do not reproduce the authors' full systems or quote their published figures.

Detector data. Table I lists the datasets. MBDD2025's train/validation/test split is unchanged throughout. For the extra-data study, the same weights were fine-tuned for 30 epochs on MBDD2025 plus dacl10k and a Roboflow corrosion set, after mapping dacl10k labels onto our five classes (wet spot and efflorescence became leakage, spalling became abscission, rust became corrosion, both crack types became crack). dacl10k's own validation split stays held out. SDNET2018 was our first choice, but every mirror we tried sat behind a bot check.

TABLE I. Datasets used.

Dataset	Role	Images
MBDD2025 [12]	Train, val, test; wall textures	10,129 / 2,894 / 1,448
dacl10k [13]	Fine-tune; held-out test	5,362 + 466; 810
Roboflow corrosion [15]	Fine-tune	851
Concrete-crack crops	Crack-presence check only	400 + 400

VI. RESULTS

A. Nine-wall yard

TABLE II. Nine-wall yard, test seeds 0–4 (confirmed reports; flight over three missions).

Planner	Prec.	Recall	F1	Flight (m)	Cov.
UW-TIG	**0.949**	**0.967**	**0.953**	**125**	**1.00**
Rückin-style	0.959	0.944	0.950	204	1.00
Alamdari-style	0.924	0.967	0.941	168	1.00
GATSBI-style	0.887	0.922	0.900	125	1.00
Random	0.921	0.837	0.873	395	1.00
Bircher-style	0.917	0.756	0.814	48	0.67
Isler-style	0.890	0.556	0.671	31	0.44

On detection quality UW-TIG is level with the best rival (F1 0.953 against 0.950 for the Rückin-style rule, with per-seed F1 for UW-TIG ranging from 0.890 to 1.000) and flies 125 m against 204 m. Its localisation error is the lowest of the seven. The planners without a coverage mechanism fail on coverage, not on detection: Isler-style and Bircher-style reach only 0.44 and 0.67 of the walls.

B. Ablations

TABLE III. Ablations: F1 and flight (m) in both yards.

Variant	F1 (9)	Flight (9)	F1 (31)	Flight (31)
Full UW-TIG	**0.953**	**125**	**0.921**	**480**
No uncertainty	0.953	125	0.924	480
No growth	0.953	125	0.921	481
No staleness	0.958	81	0.925	421
No lookahead	0.958	85	0.919	446
No coverage phase	0.680	13	0.625	181

Only the coverage phase matters: without it recall falls and coverage drops to 0.44 (nine walls) and 0.52 (31 walls). Dropping the uncertainty or growth term shifts F1 by less than 0.005 and the route by under a metre. Dropping staleness or lookahead gives a shorter route at the same F1. The uncertainty and growth terms are inactive for two reasons we could check: the coverage phase ignores them, and the scored scenarios contain no real growth, so the growth score is noise clipped at zero. Section VI-E tests both explanations directly.

C. A 31-wall yard

TABLE IV. 31-wall yard, budget 50 (F1 with 95% bootstrap interval over seeds).

Planner	F1	95% CI	Flight (m)	Cov.
UW-TIG	**0.921**	**0.906–0.938**	**480**	**1.00**
Alamdari-style	0.928	0.912–0.947	570	1.00
GATSBI-style	0.912	0.894–0.927	471	1.00
Rückin-style	0.903	0.881–0.925	860	1.00
Random	0.801	0.772–0.836	2396	0.99
Bircher-style	0.661	0.630–0.688	209	0.52
Isler-style	0.466	0.394–0.519	177	0.32

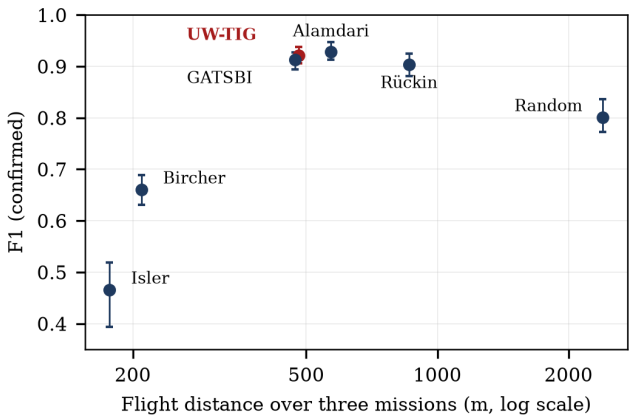


Fig. 3. F1 against flight distance in the 31-wall yard (bars: 95% bootstrap interval over seeds).

Every planner that reaches full coverage loses F1 relative to the small yard; UW-TIG goes from 0.953 to 0.921. Paired per-seed differences show UW-TIG level with the Alamdari-style rule (-0.007, interval -0.020 to +0.007) and the GATSBI-style tour (+0.009, -0.007 to +0.024), ahead of the Rückin-style rule by +0.018 (interval +0.003 to +0.035, excluding zero, 3 of 5 seeds) and far ahead of random search (+0.120). What UW-TIG keeps is cost: 480 m against 860 m for the Rückin-style rule. So at this scale UW-TIG equals the best detection scores for less flying; it does not beat every rival.

D. Training the detector on more data

TABLE V. Detector before and after fine-tuning (mAP50, precision/recall).

Test set	Original	Fine-tuned
MBDD2025 test (1,448)	0.884	0.886
dacl10k held-out (810)	0.004	0.184
dacl10k precision / recall	0.030 / 0.013	0.322 / 0.225
Crack presence, recall (800 crops)	0.005	0.035

Fine-tuning leaves the original test set unchanged (0.884 to 0.886) and lifts held-out dacl10k from 0.004 to 0.184, which is still a modest score, since dacl10k is difficult and our class mapping is a judgement call. The concrete-crack check fails for both detectors; those 227-pixel close-ups are far from anything either model was trained on, so we read it as a limit of the check, not as evidence about cracks in general.

The extra data did not help the planner. With the fine-tuned detector UW-TIG's F1 is 0.936 in the nine-wall yard (against 0.953) and 0.883 in the 31-wall yard (against 0.921). The simulator's textures are MBDD2025 photographs, so dacl10k's gain cannot show up in the loop, and fine-tuning may also have shifted the confidence calibration that the reporting rule was tuned on. We did not separate the two explanations and kept the original detector as the default.

E. Planner-only tests: oracle detector, weights and ten seeds

Two explanations remain for the null result on uncertainty and growth: the real detector gives too weak a signal, or the terms are too weak at their weights (1.5 and 2.0). We tested both. An **oracle detector** reads the scenario's ground truth and returns detections with controlled noise (15% misses, confidence rising with severity, faint defects more uncertain, defects that genuinely grow); it is a synthetic planner-only test and says nothing about real detection quality.

TABLE VI. Oracle detector, 31-wall yard (static, uncertain, multi-defect; five seeds).

Variant	F1	Flight (m)
Full UW-TIG	**0.871**	**480**
No uncertainty	0.870	480
No growth	0.871	480
No staleness	0.862	407
No lookahead	0.864	445
No coverage phase	0.613	174

With clean signals the no-uncertainty and no-growth variants still fly the identical route, and in the growing scenario every variant, including the one without the growth term, detects the growth (fraction 1.0), because the coverage and revisit logic already finds it. Raising both weights to 0, 6 and 20 leaves F1 unchanged (nine walls 0.918 in all three; 31 walls 0.822, 0.821, 0.820). Staleness and lookahead shorten the flight (nine walls 80.6 m against 124.9 m, same real-detector F1) and cost a little oracle F1, so they add efficiency, not detection quality, here.

Ten seeds. Extending the nine-wall comparison to seeds 0–9 with the real detector gives UW-TIG F1 0.933 at 125 m, Alamdari-style 0.922 at 168 m, Rückin-style 0.911 at 218 m, GATSBI-style 0.905, random 0.860, Bircher-style 0.813 and Isler-style 0.655. UW-TIG's paired advantage over the Rückin-style rule is +0.022 (95% interval −0.002 to +0.055) and

over the Alamdari-style rule +0.011 (−0.003 to +0.027): ahead on average, but not significantly. The 31-wall ten-seed run did not complete.

F. Detector size

Earlier detector rows differ in resolution and epochs, so we retrained YOLOv8n, s and m with one recipe (a fixed 4,000-image MBDD2025 subset, 640 px, 30 epochs, seed 0) and scored them on the full test split.

TABLE VII. YOLOv8 size comparison under identical training (one run each).

Model	Params (M)	mAP50	mAP50-95
YOLOv8n	3.0	0.743	0.373
YOLOv8s	11.1	0.793	0.411
YOLOv8m	25.8	0.796	0.418

Moving from n to s gains 0.050 mAP50; moving from s to m gains 0.003, within run-to-run noise, for 2.3 times the parameters. We therefore keep the small model. These subset scores are below the default detector's 0.884, which saw all 12,955 training images for 100 epochs.

VII. DISCUSSION AND LIMITATIONS

The honest summary is that UW-TIG's measured contribution is a coverage guarantee plus a cost-aware utility that is competitive with the best rule we re-implemented. Uncertainty and growth, the two ideas the planner is named for, are not visible in any number, even with a clean synthetic detector and weights up to twenty times larger (Section VI-E); we report them as untested extensions. The detector's confidence is also poorly calibrated (expected calibration error 0.29–0.33, and mean uncertainty is slightly *lower* on wrong detections), so it is used only to rank views. Recent work on uncertainty-guided crack models suggests better uncertainty is possible [19].

Limits: the baselines are re-implemented selection rules, not full systems; five seeds (ten on the nine-wall yard); the same reporting rule and a budget picked from a single pilot are used in the large yard; the 31-wall sweeps ran on an in-process re-write of the defect memory that we never compared with the database version on the same runs; walls are flat quads rendered by a software rasteriser with no wind or sensor noise; and because MBDD2025 holds no growth sequences, all temporal behaviour is tested on one procedural scenario that the detector fails to see.

VIII. CONCLUSION

We built a closed-loop inspection planner and a simulation testbed to measure it. UW-TIG matches the strongest re-implemented planners on detection in a nine-wall and a 31-wall yard at lower flight cost, and its coverage phase is necessary. Scaling the yard and training on more data both gave humbler results than the first headline, which is the reason for running them. Next we plan to texture the simulator with dacl10k-style imagery, re-tune the reporting rule for the broader detector, and add scenarios with real growth so that the uncertainty and growth terms can be tested.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering, PSG College of Technology, for its support. The code, results and figures are in the project repository.

REFERENCES

[1] J. Rückin, L. Jin, F. Magistri, C. Stachniss and M. Popović, "An informative path planning framework for active learning in UAV-based semantic mapping," *IEEE Trans. Robot.*, vol. 39, pp. 4279–4296, 2023.

[2] B. Moon *et al.*, "IA-TIGRIS: An incremental and adaptive sampling-based planner for online informative path planning," *IEEE Trans. Robot.*, 2026, arXiv:2502.15961.

[3] A. Vashisth, J. Rückin, F. Magistri, C. Stachniss and M. Popović, "Deep reinforcement learning with dynamic graphs for adaptive informative path planning," arXiv:2402.04894, 2024.

[4] H. K. Dhami, C. Reddy, V. D. Sharma, T. Williams and P. Tokekar, "GATSBI: An online GTSP-based algorithm for targeted surface bridge inspection and defect detection," arXiv:2406.16625, 2024.

[5] H. Jin, Z. Xu, H. Shen, X. Han, K. Ye and K. Shimada, "ASIP-Planner: Adaptive planning for UAV surface inspection in partially known indoor environments," in *Proc. IROS*, 2026, arXiv:2605.11119.

[6] C. Feng, Y. Xu and S. Shen, "FC-Vision: Real-time visibility-aware replanning for occlusion-free aerial target structure scanning in unknown environments," *IEEE Robot. Autom. Lett.*, 2026, arXiv:2602.13720.

[7] L. Petit and A. Lussier Desbiens, "MOAR Planner: Multi-objective and adaptive risk-aware path planning for infrastructure inspection with a UAV," in *Proc. IEEE ICRA*, 2024.

[8] S. Svystun *et al.*, "Dynamic trajectory adaptation for efficient UAV inspections of wind energy units," arXiv:2411.17534, 2024.

[9] S. Isler, R. Sabzevari, J. Delmerico and D. Scaramuzza, "An information gain formulation for active volumetric 3D reconstruction," in *Proc. IEEE ICRA*, 2016.

[10] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova and R. Siegwart, "Receding horizon 'next-best-view' planner for 3D exploration," in *Proc. IEEE ICRA*, 2016, pp. 1462–1468.

[11] S. Alamdari, E. Fata and S. L. Smith, "Persistent monitoring in discrete environments: Minimizing the maximum weighted latency between observations," *Int. J. Robot. Res.*, vol. 33, no. 1, pp. 138–154, 2014.

[12] Q. Zha, Y. Yao, Y. Zheng, W. Ma and W. Zhang, "A dataset of building surface defects collected by UAVs for machine learning-based detection," *Sci. Data*, 2025.

[13] J. Flotzinger, P. J. Rösch and T. Braml, "dacl10k: Benchmark for semantic bridge damage segmentation," in *Proc. IEEE/CVF WACV*, 2024.

[14] J. Flotzinger *et al.*, "synth-dacl: Does synthetic defect data enhance segmentation accuracy and robustness for real-world bridge inspections?" arXiv:2506.14255, 2025.

[15] F. Ciaglia, F. S. Zuppichini, P. Guerrie, M. McQuade and J. Solawetz, "Roboflow 100: A rich, multi-domain object detection benchmark," arXiv:2211.13523, 2022.

[16] S. Huang, Q. Liu, C. Chen and Y. Chen, "A real-time concrete crack detection and segmentation model based on YOLOv11," arXiv:2508.11517, 2025.

[17] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok and A. P. Schoellig, "Learning to fly – a gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control," in *Proc. IEEE/RSJ IROS*, 2021.

[18] S. Shah, D. Dey, C. Lovett and A. Kapoor, "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," in *Field and Service Robotics*, 2017.

[19] C. Li, H. He, X. Wang, W. Lin and C. H. Lim, "UnGAP: Uncertainty-guided affine prompting for real-time crack segmentation," arXiv:2605.02380, 2026.